

1) ssh hadoop@10.10.15.178

2) ls

3) cd \$HADOOP_HOME/sbin

4) ls

5) mkdir 31443

6) ls

7) cd 31443

8) ls

9) nano data.txt

Hello Welcome to PICT
Welcome to Hadoop
I like Hadoop
Welcome

10) nano WordCount.java

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            StringTokenizer itr = new StringTokenizer(value.toString());

            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {
```

```

private IntWritable result = new IntWritable();

public void reduce(Text key, Iterable<IntWritable> values,
                  Context context)
    throws IOException, InterruptedException {

    int sum = 0;

    for (IntWritable val : values) {
        sum += val.get();
    }

    result.set(sum);
    context.write(key, result);
}

}

public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");

    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setReducerClass(IntSumReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

11) Is

12) `javac -classpath `hadoop classpath` WordCount.java`

13) `jar cf WordCount.jar WordCount*.class`

14) `ls`

data.txt WordCount.class
'WordCount\$IntSumReducer.class' WordCount.jar
'WordCount\$TokenizerMapper.class' WordCount.java

15) [`start-dfs.sh`](#)

16) [`start-yarn.sh`](#)

17) `jps`

18) `hdfs mkdir hdfs://localhost:9000/`

19) `hdfs mkdir hdfs://localhost:9000/31443`

20) `hdfs mkdir hdfs://localhost:9000/31443/input`

21) `hdfs put data.txt hdfs://localhost:9000/31443/input`

**22) `hadoop jar WordCount.jar WordCount /31443/input/
31443/output`**

(single space between WordCount /31443/input/ 31443/output)

23) `hdfs ls hdfs://localhost:9000/31443/output`

24) `hdfs cat hdfs://localhost:9000/31443/output/part-r-00000`